SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
DELHI-NCR CAMPUS, MODINAGAR, GHAZIABAD

SEPM PROJECT

On

SRM CAMPUS MARKETPLACE

(2026)

Submitted By:

SL. No.	Registration Number	Student's Name	Contact no.	Department/ Section/ Year/ Semester	Signature
1	RA2311003030257	Himansh	7318486386	CSE / D/ 3/ 6	
2	RA2311003030232	Tejasvi Sharma	8690895600	CSE / D/ 3/ 6	
3	RA2311003030245	Tejasvi Verma	9084732401	CSE / D/ 3/ 6	

Subject In-Charge:
Dr. Saurabh Sharma
Assistant Professor
Dept. of CSE
SRM Institute of Science and Technology
Delhi – NCR Campus, Modinagar 201204
Ghaziabad, Uttar Pradesh, India

Lab 1: Project Identification, Business Case, and Problem Statement

1. Software Project Identification

- **Project Title:** SRM Campus Economy Platform
- **System Type:** Multi-tenant, hybrid B2B (Business-to-Consumer) and P2P (Peer-to-Peer) digital marketplace.
- **Core Architecture:** Decoupled client-server model (Web/Mobile Frontend connected to an existing asynchronous API Gateway and real-time NoSQL database).
- **Target Environment:** SRM University ecosystem.

2. Business Case

The SRM University campus operates as a massive, self-contained micro-economy. Currently, this economic potential is bottlenecked by informal, unmoderated communication channels. By deploying a centralized, strictly authenticated digital marketplace, we can formalize campus commerce and unlock significant hyper-local economic efficiency.

The platform delivers targeted value through **Asymmetric Permissions**:

- **For Students:** It provides a secure, exclusive ecosystem (verified via `@srmist.edu.in` credentials) to safely trade second-hand goods, offer freelance services (like tutoring or laundry), and make purchases without the noise of external spam.
- **For Campus Businesses:** It offers verified shops a dedicated digital storefront and business directory, providing them with a direct, professional channel to the student body to increase their daily order capacity and visibility.

Implementing this platform replaces chaotic, fragmented interactions with a structured shared economy, driving digital modernization for the university's internal commerce.

3. Problem Statement

The current state of commerce and peer-to-peer exchange within the SRM campus is highly fragmented and inefficient. Students and local businesses are forced to rely on disorganized WhatsApp groups or generic, off-campus applications to facilitate trade.

This results in several critical pain points:

1. **High Noise-to-Signal Ratio:** Students are frequently spammed by unverified local businesses in shared communication channels, degrading the user experience.
2. **Lack of Trust and Verification:** There is no centralized system to verify if a seller is an actual student or an officially sanctioned campus shop, leading to security and reliability concerns.

3. **Cross-Contamination of Markets:** The lack of distinct P2P and B2C zones means student freelance services are drowned out by commercial vendor advertisements.
4. **Inefficient Negotiation:** Bidding, negotiations, and transaction tracking are lost in endless chat threads, making it difficult for both buyers and sellers to manage orders.

A dedicated, role-based digital infrastructure is required to separate and secure these distinct economic zones while providing a unified, real-time interface for all campus transactions.

Lab 2: Analyse Stakeholder and User Description and Identify the appropriate Process Model

1. Stakeholder and User Description

The SRM Campus Economy Platform operates on a strict, role-based ecosystem. The system serves three primary user categories, governed by the "5-Door Bouncer" asymmetric permission architecture:

- **Verified Students (P2P & Consumers):**
  - *Description:* The primary user base of the platform. Access is strictly gated by the requirement of an official `@srmist.edu.in` email address.
  - *Actions:* They operate in a dual capacity. They act as consumers purchasing goods from verified shops (B2C) and as peers in the student-only zones where they can sell second-hand items, offer freelance services (e.g., tutoring, laundry), and negotiate via the real-time chat/bidding engine.
- **Verified Shops (B2C Vendors):**
  - *Description:* Local campus businesses and vendors. They do not get automatic entry; they must be officially granted the "shop" role by an administrator after an application process.
  - *Actions:* They manage official digital storefronts in the business directory, upload permanent catalog items, process student orders, and interact with the shared marketplace to view/buy student products (without the ability to post in student-exclusive zones).
- **System Administrators (Platform Overseers):**
  - *Description:* The university staff or platform managers operating behind the "God Mode Door."
  - *Actions:* They possess root-level CRUD permissions across the entire platform. Their primary responsibilities include approving pending shop applications, moderating the public feeds, taking down inappropriate listings, and banning bad actors to maintain a secure campus economy.

2. Appropriate Process Model

Agile Methodology (Iterative / Scrum)

- **Justification:** Building a complex, decoupled multi-tenant system requires an iterative approach. Because the backend (FastAPI switchboard, Firebase NoSQL, and distinct micro-services) has already been engineered in distinct phases, the frontend client must be developed and integrated incrementally.

- **Execution in Context:** * An Agile model allows the development team to build "the car around the engine" one piece at a time.
    - Sprints can be organized around specific API routers: Sprint 1 might focus on the frontend UI for `/api/users` (profile sync and authentication), Sprint 2 on `/api/products` and `/api/services` (the student zones), and Sprint 3 on `/api/chat` (the real-time negotiation engine).
    - This ensures that the strict custom FastAPI dependencies (the 5-Door security system) are rigorously tested with the client interface at each step, rather than waiting for a massive, risky integration at the end of a Waterfall cycle.

Lab 3: Software Requirements Specification (SRS) Document

1. Introduction

1.1 Purpose:

This SRS document outlines the comprehensive system, functional, and non-functional requirements for the SRM Campus Economy Platform. It serves as the definitive blueprint for the development, testing, and deployment phases, ensuring all micro-services and client interfaces adhere to the established "5-Door Bouncer" security architecture.

1.2 Scope:

The platform is a multi-tenant, role-based digital marketplace designed specifically for the SRM University ecosystem. It facilitates both peer-to-peer (P2P) student trading and business-to-consumer (B2C) transactions with verified campus shops. The system utilizes a decoupled architecture, leveraging a FastAPI backend, a real-time Firebase NoSQL database, and a responsive frontend client.

---

2. System Requirements (Operating Environment)

These are the strict technical prerequisites required to host, build, and run the platform.

2.1 Backend Server Environment:

- **Language:** Python 3.10 or higher.
- **Core Framework:** FastAPI (for asynchronous HTTP REST routing).
- **ASGI Server:** Uvicorn (to serve the FastAPI application).
- **Configuration:** pydantic-settings for strict environment variable validation (Vault management).
- **Error Tracking:** Sentry SDK integrated at the middleware level for 500-level crash reporting.

2.2 Database & Cloud Infrastructure:

- **Primary Database:** Firebase Firestore (NoSQL Document Store).

- **Authentication:** Google Cloud Identity Platform (OAuth 2.0).
- **Server Communication:** Firebase Admin SDK (for secure, server-side database CRUD operations).
- **Client Communication:** Firebase Web/Flutter SDK (for direct real-time listener mounts).

2.3 Frontend Client Environment:

- **Framework:** React.js / Next.js (for Web) or Flutter (for Cross-platform Mobile).
- **Package Manager:** npm, yarn, or flutter pub.
- **State Management:** Context API or Redux (React) / Provider or Riverpod (Flutter).

---

3. Functional Requirements (FRs)

Functional requirements define the exact behaviors, inputs, and outputs of the system, categorized by the platform's core micro-services.

3.1 Authentication & User Management (/api/users)

- **FR-1.1 (Google OAuth):** The system shall allow users to authenticate exclusively via Google standard login.
- **FR-1.2 (Domain Verification):** The backend shall parse the OAuth token and grant "Student" status *only* if the email string ends with @srmist.edu.in.
- **FR-1.3 (Profile Sync):** Upon successful login, the system shall automatically create or update the user's document in the users Firestore collection.

3.2 Student P2P Economy (/api/products & /api/services)

- **FR-2.1 (Listing Creation):** Verified students shall be able to create second-hand product listings and freelance service offerings (Title, Description, Price, Image URL).
- **FR-2.2 (Inventory Management):** Students shall be able to perform standard CRUD operations (Update Price, Mark as Sold, Delete) on their own listings.
- **FR-2.3 (B2C Isolation):** The system must completely block students from accessing or posting in the verified shop catalog directories.

3.3 Business Dashboard & Verification (/api/shops)

- **FR-3.1 (Role Application):** Standard users shall be able to submit a "Shop Verification Form" containing their business name and campus location.
- **FR-3.2 (Catalog Management):** Users possessing the explicitly verified "shop" role tag shall be able to upload permanent menu/catalog items that do not expire or enter the P2P feed.
- **FR-3.3 (Order Queue):** Verified shops shall have access to a live, real-time queue of incoming student orders.

3.4 Transaction & Order Processing (/api/orders)

- **FR-4.1 (Cart Checkout):** Students shall be able to submit a payload of selected B2C items to the backend for processing.
- **FR-4.2 (Inventory Validation):** The backend shall cross-reference the checkout payload against the shop's current database inventory before generating an order ticket.
- **FR-4.3 (Status Mutation):** The system shall allow shops to update an order's status sequentially: Pending $\rightarrow$ Processing $\rightarrow$ Ready $\rightarrow$ Completed.

3.5 Real-Time Negotiation Engine (/api/chat)

- **FR-5.1 (Room Initialization):** The system shall create a unique, isolated negotiation_room document when a student initiates a bid on a P2P listing.
- **FR-5.2 (Real-Time Sync):** The client UI must update instantly when a new message or bid is pushed to the Firestore document, bypassing the necessity to refresh the page.
- **FR-5.3 (Bid Finalization):** When a seller accepts a bid, the backend shall freeze the chat room, prevent further messages, and alter the associated product's status to "Sold".

3.6 Administrator "God Mode" (/api/admin)

- **FR-6.1 (Role Elevation):** Admins shall have the ability to read pending verification forms and mutate a user's database role from "user" to "shop".
- **FR-6.2 (Global Moderation):** Admins shall be able to forcefully delete any listing or suspend any user account across the entire platform to maintain community standards.

---

4. Non-Functional Requirements (NFRs)

Non-functional requirements dictate the system's operational constraints, focusing on security, performance, and maintainability.

4.1 Security & Access Control

- **NFR-1.1 (The 5-Door Bouncer):** Every incoming HTTP request must pass through FastAPI dependency injection to validate the JWT/OAuth token against the requested route's permission level.
- **NFR-1.2 (Secret Management):** No API keys, Firebase credentials, or secret hashes shall be hardcoded. The application must fail to start if the .env vault is missing critical variables.
- **NFR-1.3 (Database Rules):** Firestore Security Rules must be configured to reject any direct write attempts from the client that attempt to spoof uids or alter item prices.

4.2 Performance & Scalability

- **NFR-2.1 (Asynchronous I/O):** The FastAPI server must define all endpoint functions using async def to ensure the event loop is not blocked during database read/write operations.
- **NFR-2.2 (Latency Target):** Standard REST API endpoints (e.g., fetching a user profile) should return a response in under 200ms.
- **NFR-2.3 (Real-Time Optimization):** Client-side WebSocket listeners for the chat engine must be explicitly detached/unmounted when the user navigates away from the chat screen to prevent memory leaks and database quota exhaustion.

4.3 Reliability & Error Handling

- **NFR-3.1 (Native Graceful Degradation):** The API must handle anticipated logic errors (e.g., "Item out of stock", "Invalid Email") using native Python try/except blocks, returning structured HTTP 400-level JSON responses to the client.
- **NFR-3.2 (Crash Auditing):** Unhandled system crashes (HTTP 500) must be automatically routed to Sentry with the exact stack trace and timestamp, without relying on heavy third-party wrapper libraries that clutter the core routing code.

4.4 Maintainability

- **NFR-4.1 (Modular Routing):** The backend codebase must not exist in a single main.py file. It must utilize FastAPI's APIRouter to strictly separate user, shop, product, and chat logic into distinct modular files.
- **NFR-4.2 (API Contract):** The backend must automatically generate and maintain an interactive OpenAPI (Swagger) specification at the /docs route to serve as the single source of truth for frontend integration.
-

Lab 4: Project Plan, Job Roles, and Effort Calculation (Lean Startup Model)

1. Project Plan (Based on Scope)

Since the core backend architecture (FastAPI, Firebase, 5-Door Bouncer security) is already established, this project plan focuses exclusively on the rapid development, integration, and deployment of the client interfaces over an 8-week sprint cycle.

- **Phase 1: Frontend Initialization & The Student Economy (Weeks 1–3)**
  - Initialize the React/Next.js (or Flutter) client environment.
  - Build the authentication flow connecting to the /api/users endpoint.
  - Develop the Student Portal (P2P feeds, adding items, shopping cart).
- **Phase 2: The Business Zone & God Mode (Weeks 4–5)**
  - Develop the Shop Dashboard for verified vendors to manage inventory.
  - Build the Admin Control Panel for shop verification and system moderation.
- **Phase 3: Real-Time Integration & Testing (Weeks 6–7)**
  - Wire up the /api/chat negotiation engine with real-time Firestore listeners.
  - Conduct role-based integration testing to ensure the asymmetric permissions (e.g., shops cannot post in student zones) hold up on the client side.
- **Phase 4: Hardening & Campus Beta Launch (Week 8)**
  - Verify native exception handling and Sentry crash reporting integrations.
  - Final deployment of the frontend client.
  - Go-live for the SRM Campus Beta.

2. Job Roles and Responsibilities

With a compact team of three, roles must be highly specialized and focused purely on execution.

Role	Key Responsibilities

Backend & Security Architect	Maintains the FastAPI switchboard, resolves API bottlenecks, and ensures the 5-Door permission logic remains secure during frontend integration.
Frontend UI/UX Engineer	Executes the visual interfaces for Students, Shops, and Admins using established component libraries (e.g., Tailwind/Material UI) to maximize speed.
Database & DevOps Engineer	Optimizes Firestore NoSQL document queries, manages real-time chat latency, and oversees the final cloud deployment and standard Python logging.

3. Project Effort Calculation

This calculation reflects a highly disciplined, "sniper approach" to software engineering, typical of a student incubation project or lean startup, rather than a traditional corporate bloat model.

Assumptions & Parameters:

- **Team Size:** 3 Core Engineers.
- **Project Duration:** 8 Weeks (Accelerated Launch).
- **Working Capacity:** 6 Hours of highly focused development time per week, per engineer.

Calculation:

- **Weekly Team Effort:** 3 Engineers × 6 Hours = 18 Hours / Week.
- **Total Estimated Effort:** 18 Hours / Week × 8 Weeks = **144 Person-Hours**.

Strategic Execution Note: With exactly 144 hours to ship, the team will rely heavily on the pre-built FastAPI Swagger UI (/docs) as the API contract, allowing the frontend engineer to move rapidly without waiting on backend clarifications.

Lab 5: Prepare the Work Breakdown Structure (WBS) and Risk Plan

1. Work Breakdown Structure (8-Week Lean Sprint)

This WBS maps the integration of the client interfaces to the already-built FastAPI and Firebase backend. It is optimized for a 3-person specialized team.

- **Phase 1: Foundation & Authentication (Weeks 1–2)**
  - **1.1** Scaffold the Frontend client (React/Next.js or Flutter) and establish standard UI component libraries.
  - **1.2** Connect the Google standard login to the /api/users endpoint for automated profile syncing.
  - **1.3** Implement client-side routing guards to mirror the backend's "5-Door Bouncer" logic (Student, Shop, Admin).
- **Phase 2: The Student Economy (P2P) (Weeks 3–4)**
  - **2.1** Develop the Student Dashboard and global marketplace feed.
  - **2.2** Integrate /api/products and /api/services endpoints to allow verified @srmist.edu.in users to post items and freelance offerings.
  - **2.3** Build the shopping cart and basic checkout UI flow.
- **Phase 3: Business Zone & Real-Time Engine (Weeks 5–6)**
  - **3.1** Build the Shop Dashboard, allowing verified vendors to execute CRUD operations on their permanent catalogs via /api/shops.
  - **3.2** Wire up the /api/chat real-time negotiation engine, attaching Firestore listeners for instant bidirectional messaging and bidding.
  - **3.3** Develop the Admin "God Mode" interface for reviewing shop applications and moderating the global feed.
- **Phase 4: Hardening & Beta Launch (Weeks 7–8)**
  - **4.1** Conduct role-based penetration testing to ensure frontend state manipulation cannot bypass the backend's asymmetric permissions.
  - **4.2** Verify native client-side error boundaries and confirm that backend Sentry logging catches any 500-level API crashes gracefully.
  - **4.3** Finalize environment vault variables and execute the SRM Campus Beta production deployment.

2. Risk Identification and Mitigation Plan

With highly constrained working hours (6 hours per engineer per week), the project faces unique operational risks.

Risk Category	Potential Impact	Mitigation Strategy
Schedule / Scope Creep	**High:** Attempting to build non-essential features (e.g., complex recommendation algorithms) will immediately break the 8-week deadline.	**Strict Scope Lock:** Defer all "nice-to-have" features to Post-Beta updates. The team must focus strictly on the core P2P/B2C marketplace and role-based login functionality.
Frontend/Backend Desync	**High:** UI engineers wasting limited hours waiting for backend clarifications or endpoint updates.	**API Contract Enforcement:** Utilize the pre-built FastAPI Swagger UI (/docs) as the absolute single source of truth. No core backend architecture changes are permitted during this phase.
Database Cost / Quota Spikes	**Medium:** Inefficient real-time listeners in the chat or global feed could drain the Firebase free tier rapidly during testing.	**Query Optimization:** Paginate the product feeds (e.g., load 20 items at a time) and limit active WebSocket/Firestore listeners strictly to open chat rooms, not the global state.
System Instability	**Medium:** Uncaught frontend exceptions causing the app to white-screen or freeze.	**Native Guardrails:** Implement robust React/Flutter error boundaries and ensure every API call utilizes standard try/except (or try/catch) logic to display user-friendly error messages upon failure.

With the schedule locked and the risks mitigated, we are ready to move to the visual blueprints.

Lab 6: System Architecture, Use Case Diagram, and ER Diagram Descriptions

1. System Architecture Description

The SRM Campus Economy Platform is built on a **Decoupled 3-Tier Serverless/Cloud-Native Architecture**. This design physically separates the user interfaces from the core business logic and database, ensuring high scalability and allowing the distinct client portals to evolve independently of the backend engine.

- **Tier 1: Presentation Layer (Client Applications)**
  - This tier consists of the frontend interfaces (e.g., React, Next.js, or Flutter) operating on user devices.
  - It contains three distinct environments: the **Student Mobile App** (optimized for browsing and chatting), the **Shop Web Dashboard** (optimized for managing inventory and incoming orders), and the **Admin Portal** (for platform moderation).
  - It communicates with the backend entirely via stateless RESTful HTTP requests, except for real-time negotiation features, which open direct WebSocket connections to the database layer.
- **Tier 2: Application Layer (FastAPI Gateway & Security)**
  - Hosted on cloud infrastructure, this is the central switchboard of the application.
  - Every incoming REST request first hits the **5-Door Bouncer Security Logic** (custom FastAPI dependencies). This layer authenticates the token and routes traffic based on asymmetric permissions (Student Door, Shop Door, God Mode Door).
  - Once cleared, requests are processed by modular API micro-services (`/api/users`, `/api/products`, `/api/shops`, `/api/orders`) which execute the core business logic, handle native exception management, and format the data payloads.
- **Tier 3: Data & Cloud Infrastructure Layer (Firebase Firestore)**
  - A NoSQL cloud database that serves as the single source of truth for persistent storage.
  - To ensure sub-second latency for the platform's chat and bidding engine, this tier utilizes **Real-time Firestore Listeners**. The client applications bypass the

FastAPI switchboard for chat updates, hooking directly into the database to sync messages instantly without polling the application layer.

2. Use Case Diagram Description

The platform relies on strict role-based access control. The use case boundaries ensure that the distinct economic zones (P2P and B2C) do not overlap inappropriately.

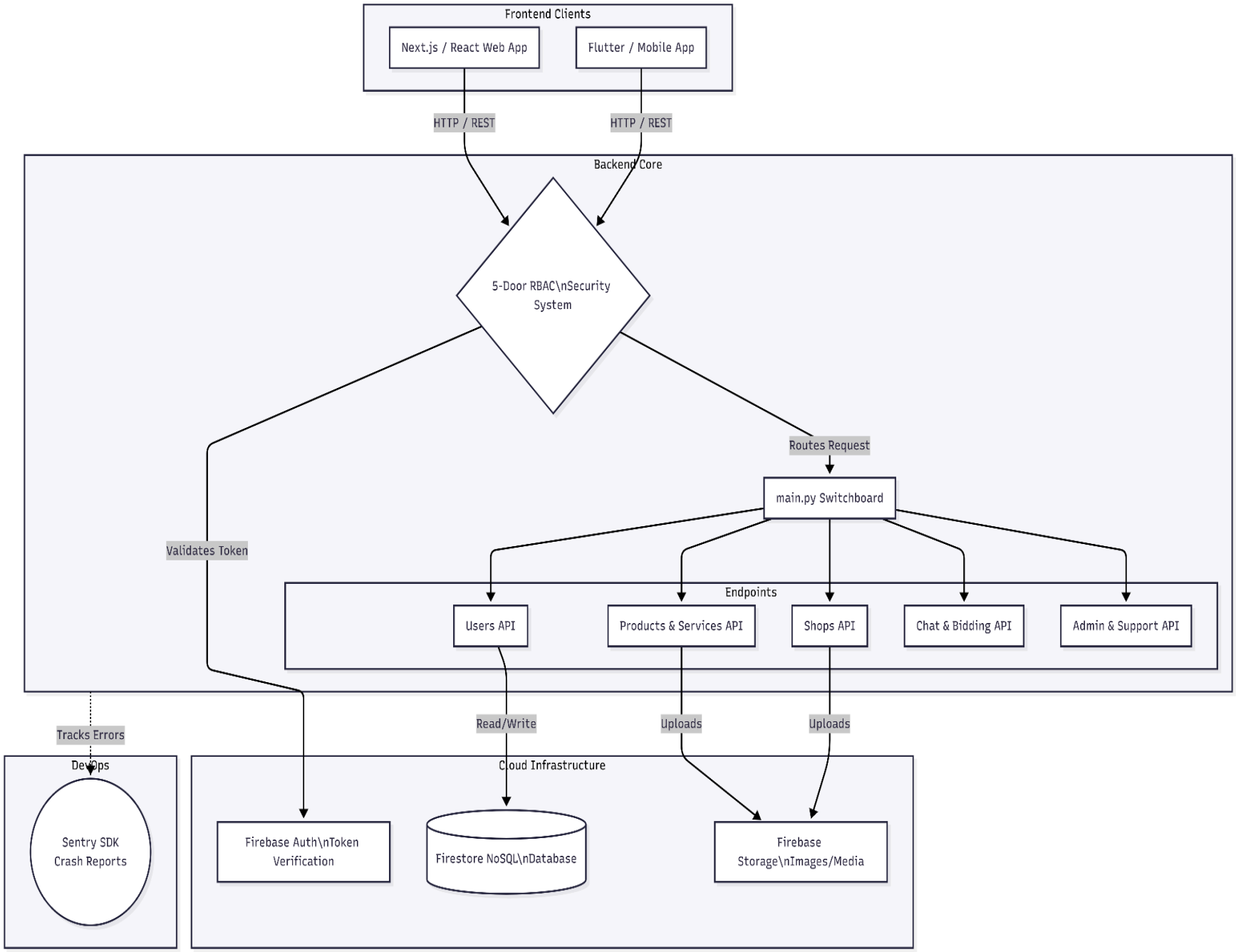
- **Actor: Verified Student**
  - **Authentication:** Must log in using an official `@srmist.edu.in` Google Workspace account.
  - **Core Use Cases:** Browse the global marketplace feed, manage a personal shopping cart, and place orders with verified campus shops.
  - **P2P Use Cases:** Post second-hand items for sale, offer freelance campus services (e.g., tutoring), and initiate direct chat/bids with other users.
  - **System Constraints:** Strictly prohibited from accessing business dashboard endpoints or posting in the B2C permanent catalog zones.
- **Actor: Verified Shop**
  - **Authentication:** Logs in via Google, but must have their `role` explicitly elevated to "shop" by an administrator to access vendor features.
  - **B2C Use Cases:** Manage a digital storefront (perform CRUD operations on catalog items/menus), view the live order queue, accept new orders, and update order fulfillment statuses.
  - **System Constraints:** Permitted to browse the global feed, but explicitly blocked from posting listings in the student-only P2P zones.
- **Actor: System Admin (God Mode)**
  - **Core Use Cases:** Review and approve/reject pending "Shop Verification" applications submitted by local vendors.
  - **Moderation Use Cases:** Suspend or ban bad actors, instantly delete inappropriate listings globally, and view high-level system analytics.

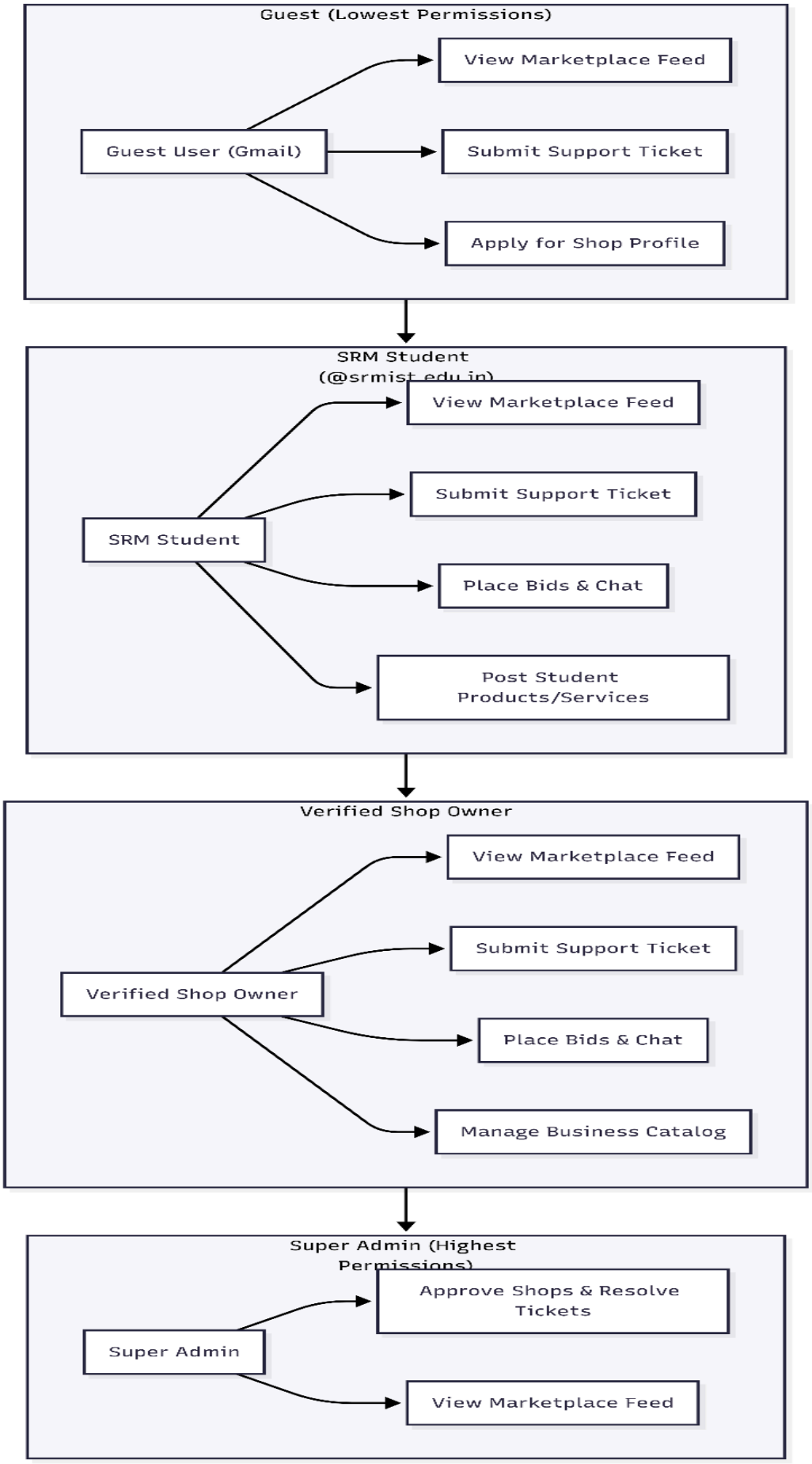
3. ER / Database Schema Description (NoSQL Document Structure)

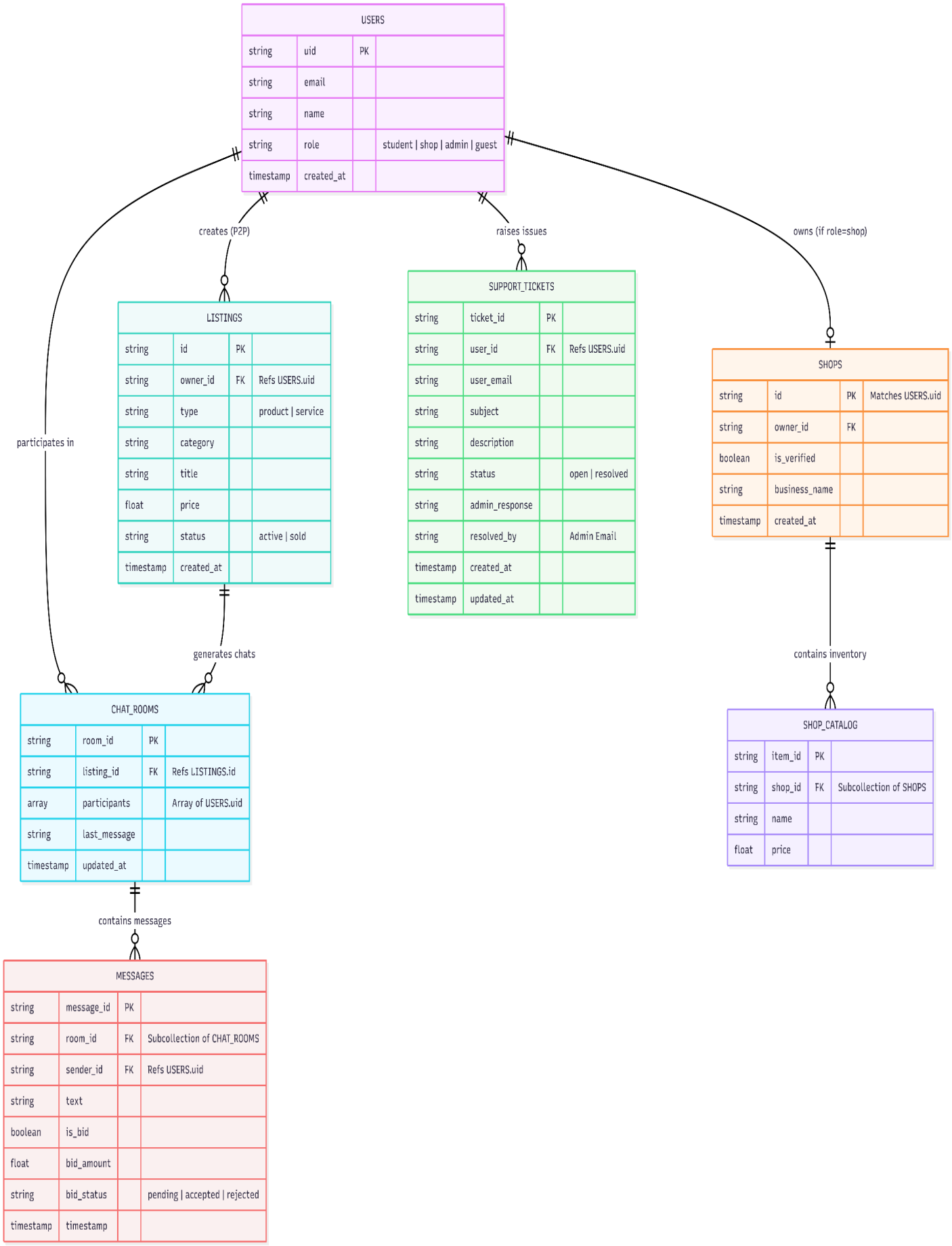
Because the platform leverages Firebase Firestore, the database relies on a highly flexible, horizontally scalable Document-Collection schema rather than traditional relational SQL tables. Data is denormalized where necessary to prioritize fast read speeds.

- **Collection: `users`**
  - Acts as the central identity registry.
  - *Key Fields:* `uid` (Primary Key linked to Auth Token), `email`, `role` (String constraint: "student", "shop", "admin"), `display_name`, and `created_at`.
- **Collection: `listings`**
  - A unified collection storing both B2C shop catalogs and P2P student items, distinguishable by their type.

    - *Key Fields:* `listing_id` (Primary Key), `owner_id` (Foreign Key referencing `users.uid`), `title`, `price`, `type` ("b2c_product", "p2p_item", "p2p_service"), and `status` ("active", "sold", "hidden").
- **Collection: `orders`**
    - Tracks the lifecycle of a B2C transaction.
    - *Key Fields:* `order_id` (Primary Key), `student_id` (Foreign Key for the buyer), `shop_id` (Foreign Key for the vendor), `cart_items` (Array of listing references), `total_value`, and `order_status` ("pending", "processing", "ready", "completed").
- **Collection: `negotiation_rooms`**
    - The backend structure powering the real-time chat and bidding engine.
    - *Key Fields:* `room_id` (Primary Key), `participants` (Array containing exactly two `users.uid` strings), `context_item` (Foreign Key referencing the `listing_id` being discussed), `latest_bid`, and a sub-collection for the individual timestamped `messages`.


Frontend Clients
Next.js / React Web App
Flutter / Mobile App
HTTP / REST
HTTP / REST
Backend Core
5-Door RBAC\nSecurity System
Routes Request
main.py Switchboard
Validates Token
Endpoints
Users API
Products & Services API
Shops API
Chat & Bidding API
Admin & Support API
Tracks Errors
Read/Write
Uploads
Uploads
DevOps
Sentry SDK Crash Reports
Cloud Infrastructure
Firebase Auth\nToken Verification
Firestore NoSQL\nDatabase
Firebase Storage\nImages/Media


Guest (Lowest Permissions)
Guest User (Gmail)
View Marketplace Feed
Submit Support Ticket
Apply for Shop Profile
SRM Student
(@srmist.edu.in)
SRM Student
View Marketplace Feed
Submit Support Ticket
Place Bids & Chat
Post Student Products/Services
Verified Shop Owner
Verified Shop Owner
View Marketplace Feed
Submit Support Ticket
Place Bids & Chat
Manage Business Catalog
Super Admin (Highest Permissions)
Super Admin
Approve Shops & Resolve Tickets
View Marketplace Feed


USERS
string uid PK
string email
string name
string role student | shop | admin | guest
timestamp created_at
creates (P2P)
raises issues
owns (if role=shop)
participates in
LISTINGS
string id PK
string owner_id FK Refs USERS.uid
string type product | service
string category
string title
float price
string status active | sold
timestamp created_at
SUPPORT_TICKETS
string ticket_id PK
string user_id FK Refs USERS.uid
string user_email
string subject
string description
string status open | resolved
string admin_response
string resolved_by Admin Email
timestamp created_at
timestamp updated_at
SHOPS
string id PK Matches USERS.uid
string owner_id FK
boolean is_verified
string business_name
timestamp created_at
generates chats
contains inventory
CHAT_ROOMS
string room_id PK
string listing_id FK Refs LISTINGS.id
array participants Array of USERS.uid
string last_message
timestamp updated_at
SHOP_CATALOG
string item_id PK
string shop_id FK Subcollection of SHOPS
string name
float price
contains messages
MESSAGES
string message_id PK
string room_id FK Subcollection of CHAT_ROOMS
string sender_id FK Refs USERS.uid
string text
boolean is_bid
float bid_amount
string bid_status pending | accepted | rejected
timestamp timestamp

Here is the detailed, text-only documentation for Lab 7.

Since you have already visualized the diagrams, this documentation provides the exact structural and behavioral descriptions needed to back them up. It perfectly maps your data flows to the decoupled FastAPI/Firebase architecture and translates your NoSQL entities into conceptual Object-Oriented application classes.

Lab 7: Data Flow Diagram (DFD) and Class Diagram Descriptions

1. Data Flow Diagram (DFD) Description

The DFD maps how information moves through your system, from the moment a user clicks a button on the client application to the moment the database registers the change.

A. Level 0 DFD (Context Diagram)

The Level 0 diagram represents the entire "SRM Campus Economy Platform" as a single, central process interacting with its external environment (the users).

- **Central Process:** The SRM Campus API Gateway.
- **External Entity 1: Verified Student**
  - *Data Flowing In:* Google Auth Token (`@srmist.edu.in`), P2P listing details, Cart checkout data, Chat messages, and Bids.
  - *Data Flowing Out:* Rendered global marketplace feeds, real-time chat updates, order status notifications.
- **External Entity 2: Verified Shop**
  - *Data Flowing In:* Google Auth Token, Vendor application documents, Catalog inventory updates (prices, stock), Order state changes (e.g., "Accepted").
  - *Data Flowing Out:* Live incoming order queues, verification status updates.
- **External Entity 3: System Admin**
  - *Data Flowing In:* Admin credentials, Moderation commands (Delete post, Ban user, Approve shop).
  - *Data Flowing Out:* System logs, pending shop application queues.

B. Level 1 DFD (Process Decomposition)

The Level 1 diagram breaks the central API Gateway into four major sub-processes, directly mapping to your FastAPI routers.

- **Process 1.0: Security & Routing (The 5-Door Bouncer)**
  - *Input:* Raw HTTP requests with Auth Tokens from any client.

    - *Action:* Decodes the token, checks the `users` database for role tags, and either rejects the request (HTTP 403) or routes it to the correct micro-service.
- **Process 2.0: Catalog & Feed Management (`/api/products`, `/api/shops`)**
    - *Input:* Verified requests to create, read, update, or delete items.
    - *Action:* Separates logic based on the user role. It writes B2C items to the shop's permanent catalog and P2P items to the temporary student feed.
    - *Data Store:* Reads/Writes to the `listings` database collection.
- **Process 3.0: Transaction & Order Processing (`/api/orders`)**
    - *Input:* Shopping cart payloads from Students; Status updates from Shops.
    - *Action:* Validates shop inventory, calculates totals, and creates a pending order ticket. Notifies the shop client.
    - *Data Store:* Reads/Writes to the `orders` database collection.
- **Process 4.0: Real-Time Negotiation Engine (`/api/chat`)**
    - *Input:* Bids and chat strings from Students.
    - *Action:* This process largely bypasses the standard REST HTTP flow, utilizing direct WebSocket/Firestore SDK writes to push messages to all participants instantly.
    - *Data Store:* Reads/Writes to the `negotiation_rooms` database collection.

---

2. Class Diagram Description (OOPS Mapping)

Even though the platform uses a NoSQL database (Firebase), the application logic heavily relies on Object-Oriented Programming (OOP). In your FastAPI backend, these classes are implemented as **Pydantic Models** (for data validation) and **Python Controllers** (for business logic). On the frontend, they act as state management classes.

Here is the conceptual breakdown of the primary classes, their attributes, and their operational methods:

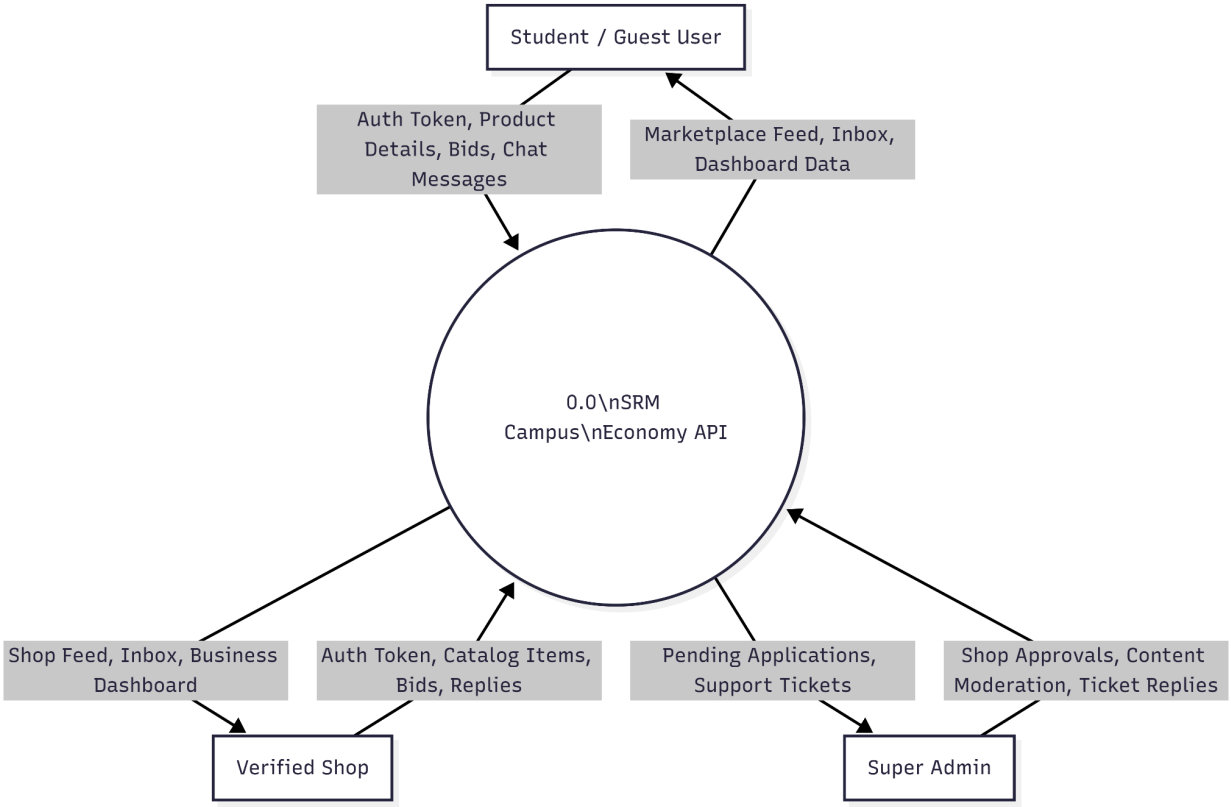
A. The User Hierarchy (Inheritance)

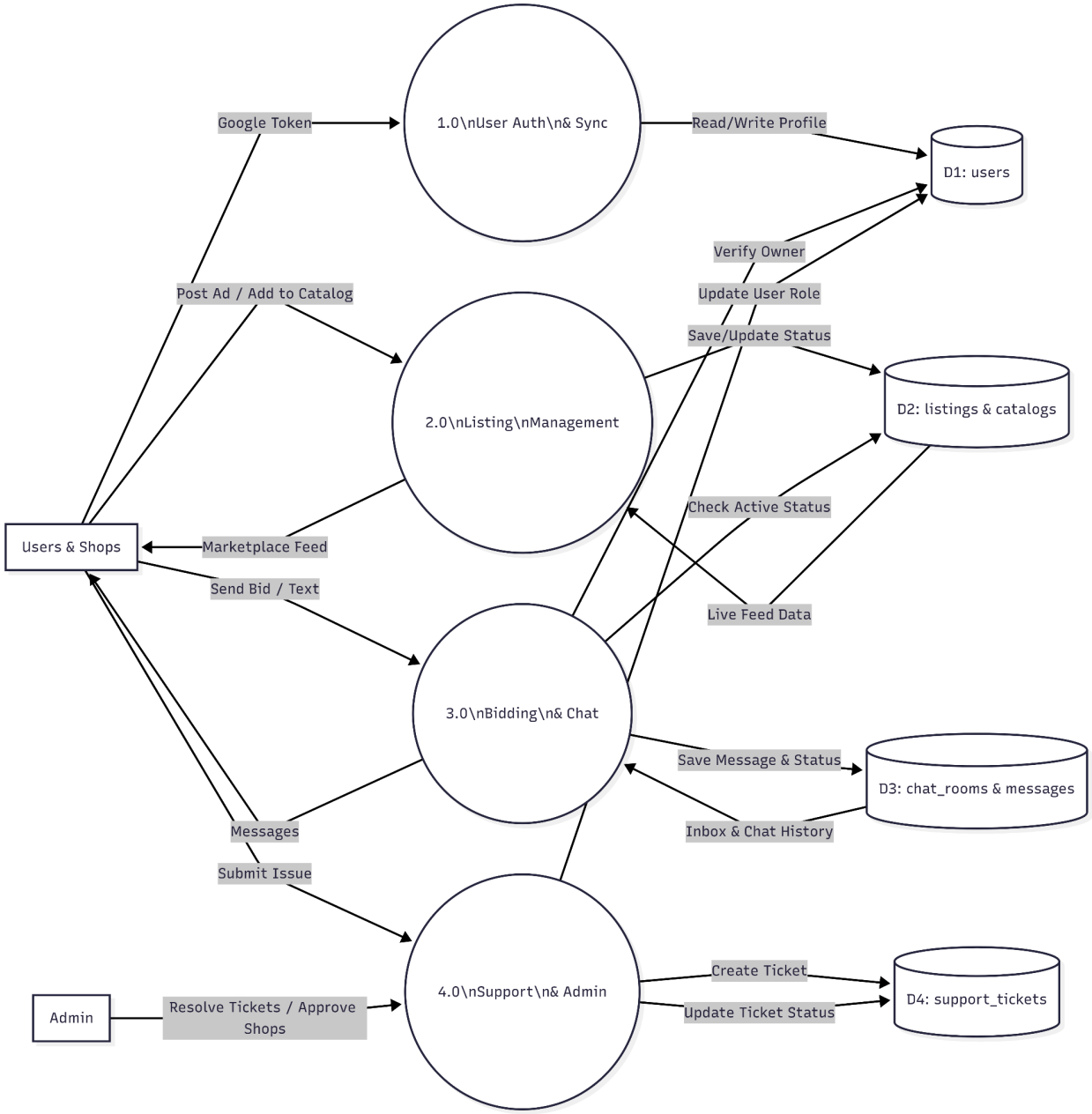
- **Class: `Account` (Base Class)**
    - *Attributes:* `uid` (string), `email` (string), `displayName` (string), `role` (enum).
    - *Methods:* `authenticateToken()`, `syncProfile()`, `logout()`.
- **Class: `Student` (Inherits from `Account`)**
    - *Specific Attributes:* `srm_id` (derived from email).
    - *Specific Methods:* `createP2PListing()`, `addToCart()`, `checkoutCart()`, `initiateBid()`.
- **Class: `Shop` (Inherits from `Account`)**

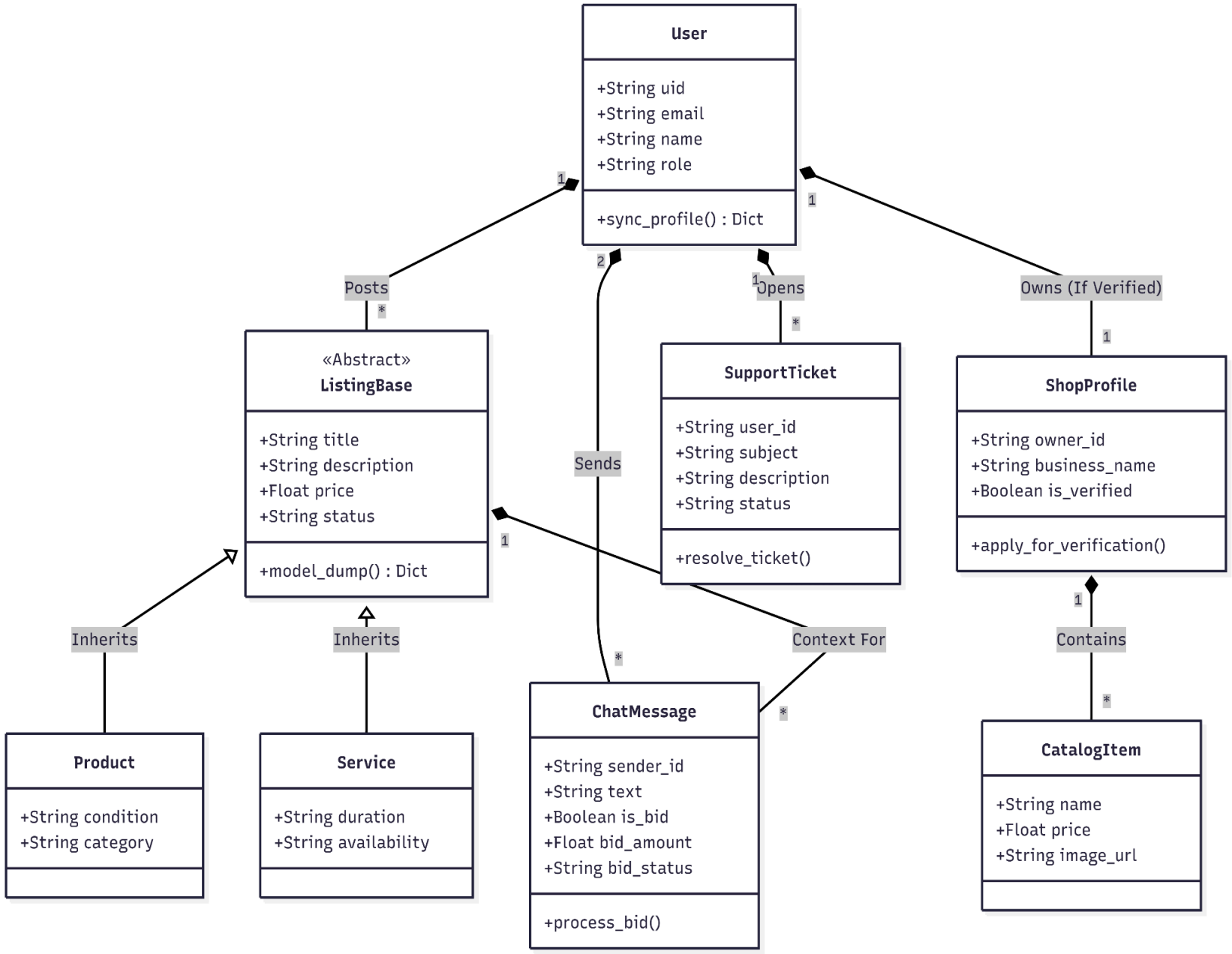
    - *Specific Attributes:* `businessName` (string), `operatingHours` (string), `isVerified` (boolean).
    - *Specific Methods:* `submitVerificationForm()`, `addCatalogItem()`, `acceptOrder()`, `markOrderReady()`.
- **Class: `Admin` (Inherits from `Account`)**
    - *Specific Methods:* `approveShopRole()`, `deleteGlobalListing()`, `suspendAccount()`.

B. The Economy Classes

- **Class: `Listing`**
    - *Attributes:* `listingId` (string), `ownerId` (string), `title` (string), `price` (float), `type` (enum: B2C, P2P), `status` (enum: Active, Sold).
    - *Methods:* `updatePrice()`, `toggleVisibility()`, `markAsSold()`.
- **Class: `Order`**
    - *Attributes:* `orderId` (string), `studentId` (string), `shopId` (string), `items` (List[Listing]), `totalAmount` (float), `orderStatus` (enum).
    - *Methods:* `calculateTotal()`, `validateInventory()`, `updateStatus()`.
- **Class: `NegotiationRoom`**
    - *Attributes:* `roomId` (string), `participantIds` (List[string]), `contextItemId` (string), `latestBid` (float), `isActive` (boolean).
    - *Methods:* `broadcastMessage()`, `registerNewBid()`, `lockRoomOnAcceptance()`.


Student / Guest User
Auth Token, Product Details, Bids, Chat Messages
Marketplace Feed, Inbox, Dashboard Data
0.0\nSRM Campus\nEconomy API
Shop Feed, Inbox, Business Dashboard
Auth Token, Catalog Items, Bids, Replies
Pending Applications, Support Tickets
Shop Approvals, Content Moderation, Ticket Replies
Verified Shop
Super Admin


Google Token
1.0\nUser Auth\n& Sync
Read/Write Profile
D1: users
Verify Owner
Update User Role
Post Ad / Add to Catalog
Save/Update Status
2.0\nListing\nManagement
D2: listings & catalogs
Check Active Status
Users & Shops
Marketplace Feed
Send Bid / Text
Live Feed Data
3.0\nBidding\n& Chat
Save Message & Status
D3: chat_rooms & messages
Inbox & Chat History
Messages
Submit Issue
4.0\nSupport\n& Admin
Create Ticket
D4: support_tickets
Update Ticket Status
Admin
Resolve Tickets / Approve Shops


User
+String uid
+String email
+String name
+String role
+sync_profile() : Dict
Posts
Opens
Owns (If Verified)
Sends
«Abstract»
ListingBase
+String title
+String description
+Float price
+String status
+model_dump() : Dict
SupportTicket
+String user_id
+String subject
+String description
+String status
+resolve_ticket()
ShopProfile
+String owner_id
+String business_name
+Boolean is_verified
+apply_for_verification()
Inherits
Inherits
Context For
Contains
Product
+String condition
+String category
Service
+String duration
+String availability
ChatMessage
+String sender_id
+String text
+Boolean is_bid
+Float bid_amount
+String bid_status
+process_bid()
CatalogItem
+String name
+Float price
+String image_url

Lab 8: Interaction, State Chart, and Activity Diagram Descriptions

1. Interaction Diagram Description (Communication Flow)

While Sequence diagrams focus on time, Communication (or Collaboration) diagrams focus on the structural organization of the objects sending and receiving messages.

Scenario: A Student placing a B2C Order with a Campus Shop

This maps the journey of a payload through your "5-Door Bouncer" and decoupled backend.

- **Object 1: Student Client (Mobile/Web)**
  - *Message Out:* Sends an HTTP POST request (checkoutPayload) to the API Gateway.
  - *Message In:* Receives the HTTP 201 Created response and updates the UI cart.
- **Object 2: FastAPI Gateway (main.py Router)**
  - *Message Out:* Intercepts the request and forwards the token to the Security Dependency.
- **Object 3: Security Dependency (The "Student Door")**
  - *Message Out:* Validates the token and confirms the @srmist.edu.in domain. Returns a Pass or Fail boolean back to the Gateway.
- **Object 4: Order Controller (/api/orders)**
  - *Message In:* Receives the validated payload from the Gateway.
  - *Message Out:* Queries the database: checkInventory(shop_id, items).

- **Object 5: Firebase Database (Firestore)**
  - *Message In:* Receives the inventory check. Returns stock_status.
  - *Message In:* If stock is valid, receives the command from the Order Controller to createDocument(order_ticket).
- **Object 6: Shop Dashboard Client**
  - *Message In:* Because this client is hooked into a Real-Time Firestore Listener, it instantly receives a document_added event from the Database and flashes a "New Order" notification on the vendor's screen, bypassing the FastAPI server entirely for the push notification.

2. State Chart Diagram Description

State charts track the lifecycle of a single entity. In your campus economy, the most critical entity with a dynamic lifecycle is the **B2C Order**.

Entity: The Order Document

- **Initial State [Start]** $\rightarrow$ Transitions immediately to **Pending**.
  - *Trigger:* Student successfully checks out. Document is created in Firestore.
- **State: Pending**
  - *Transition A:* $\rightarrow$ **Processing**.
    - *Trigger:* The Shop vendor clicks "Accept Order" on their dashboard.
  - *Transition B (Alternative):* $\rightarrow$ **Cancelled**.
    - *Trigger:* The Shop vendor clicks "Reject" (e.g., out of stock), or the Student cancels before acceptance.
- **State: Processing**
  - *Transition:* $\rightarrow$ **Ready**.
    - *Trigger:* The Shop vendor finishes preparing the food/item and clicks "Mark as Ready." This triggers a notification on the Student's phone.
- **State: Ready**
  - *Transition:* $\rightarrow$ **Completed**.
    - *Trigger:* The Student picks up the order and the vendor clicks "Handed Over," closing the transaction loop.
- **Final State [End]**
  - Reached when the order hits either **Completed** or **Cancelled**. The document is archived.

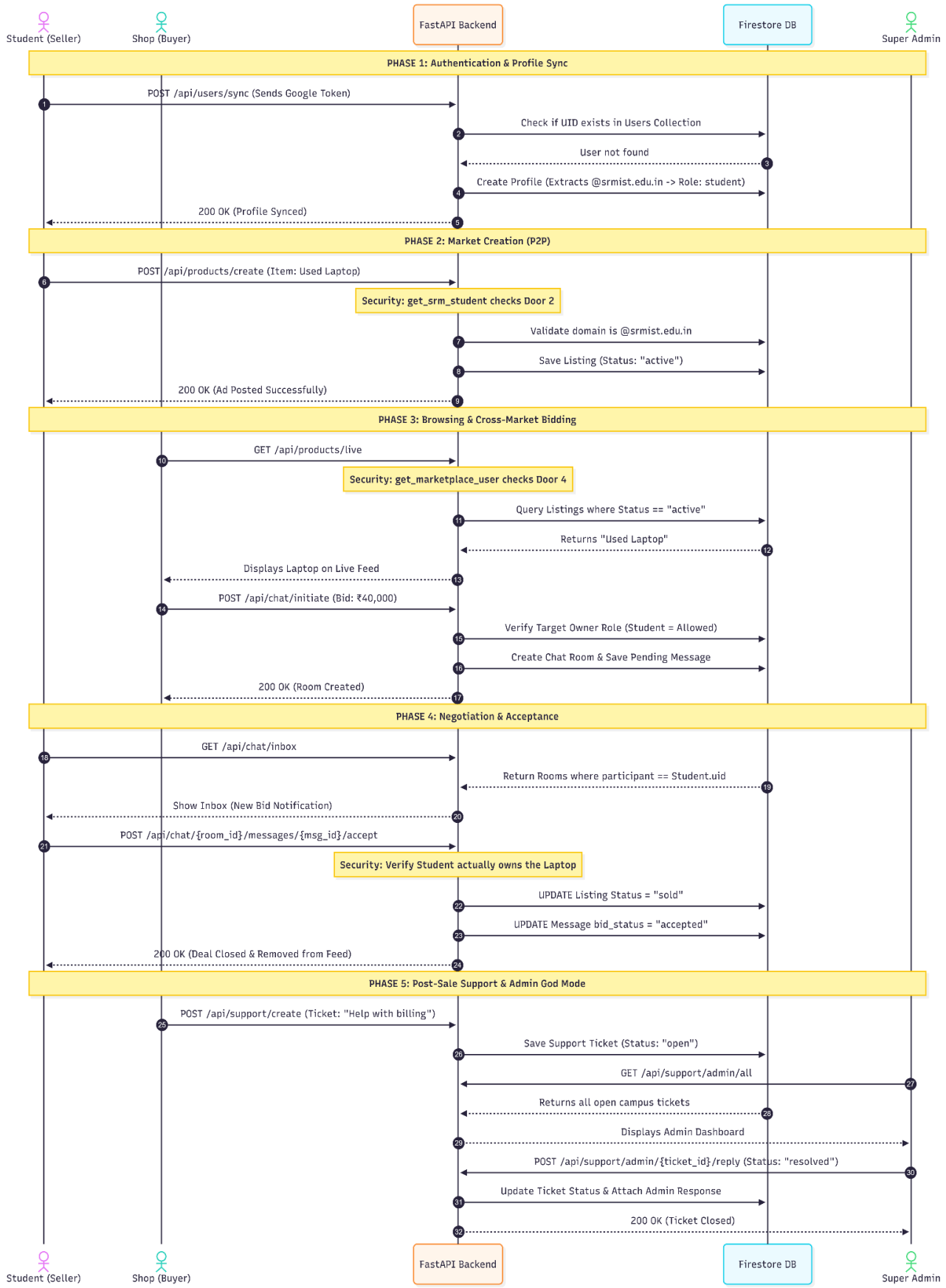
3. Activity Diagram Description

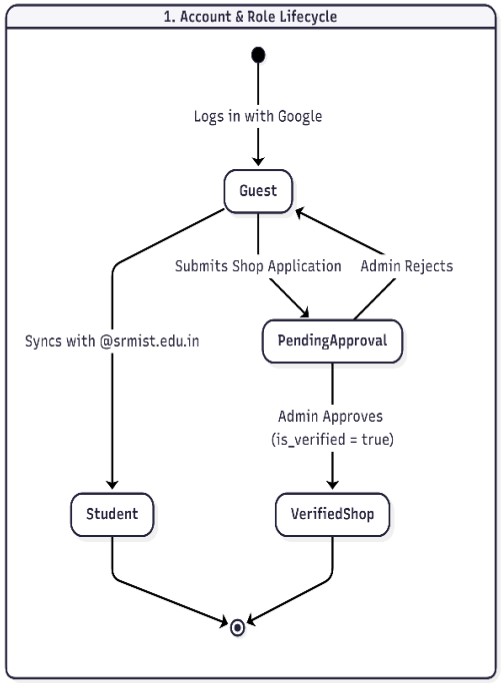
Activity diagrams map out the sequential and concurrent flow of a complex business process. This description maps out your strict role-elevation process.

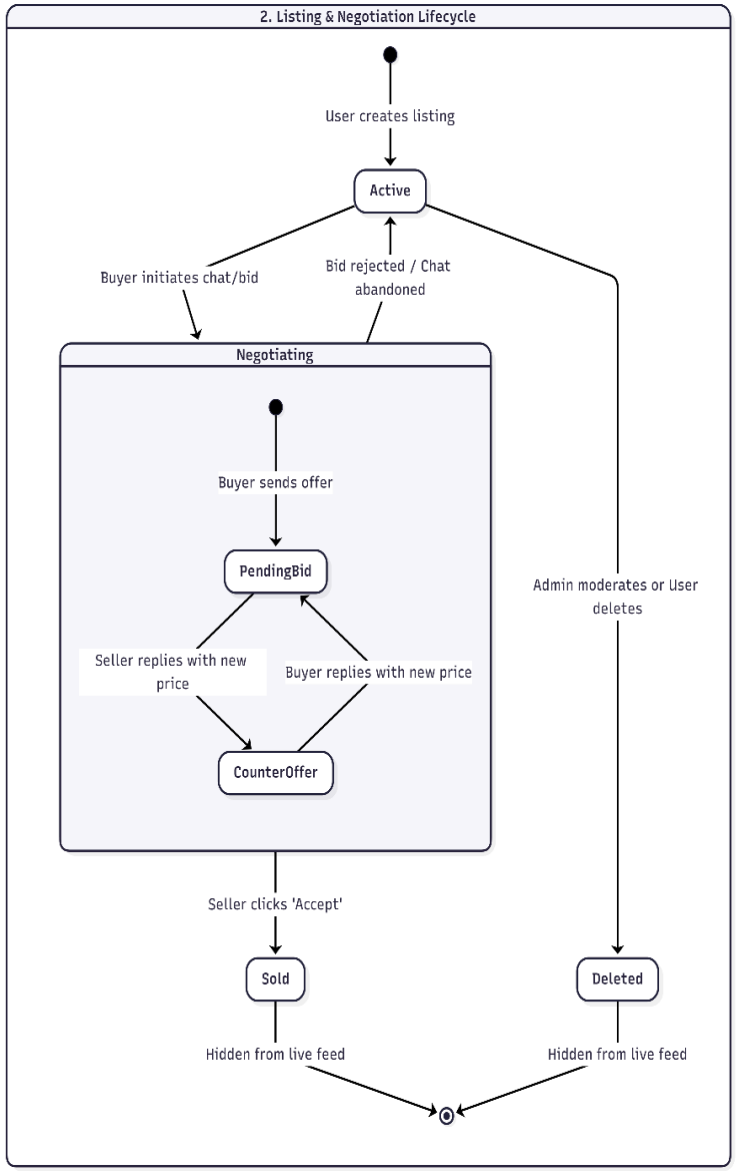
Process: Shop Verification and Catalog Access (Swimlane Format)

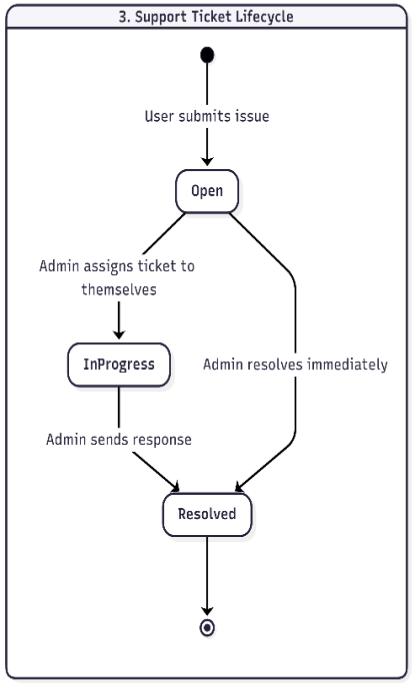
- **Swimlane 1: The Vendor (Client UI)**
  - *Start Node:* Vendor logs into the app using a standard Google account.

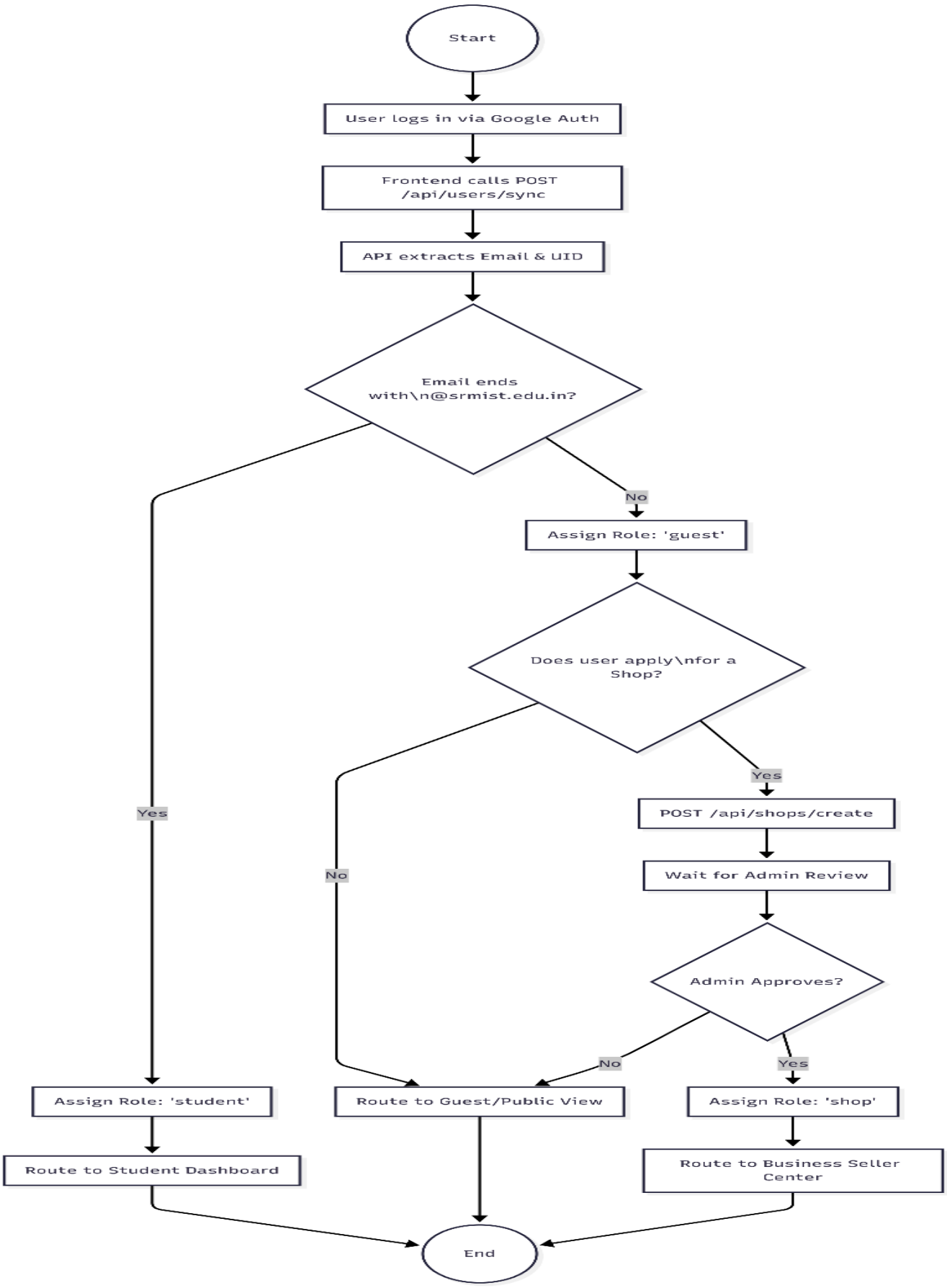
    - *Action:* Vendor attempts to access /api/shops.
    - *System Interrupt:* Denied. User is redirected to the "Verification Form."
    - *Action:* Vendor fills out business details and submits the form.
- **Swimlane 2: The System (FastAPI / Firebase)**
    - *Action:* The backend creates a pending_verification ticket in the database.
    - *Action:* System places the vendor in a "Waiting" UI state.
- **Swimlane 3: The Administrator (God Mode Portal)**
    - *Action:* Admin logs in, passing the God Mode Door dependency.
    - *Action:* Admin views the queue of pending shop applications.
    - *Decision Node:* Does the business meet campus guidelines?
        - *Branch A [No]:* Admin clicks "Reject." System emails the vendor. Process ends.
        - *Branch B [Yes]:* Admin clicks "Approve."
- **Swimlane 2: The System (FastAPI / Firebase) - *Resumed***
    - *Action:* System executes a root-level update, changing the vendor's database role from "guest" to "shop".
- **Swimlane 1: The Vendor (Client UI) - *Resumed***
    - *Action:* Vendor refreshes the app. The UI detects the new "shop" role tag.
    - *Action:* Vendor successfully passes the "Shop Door" dependency.
    - *Final Node:* Vendor accesses the Dashboard and uploads their first permanent catalog item.

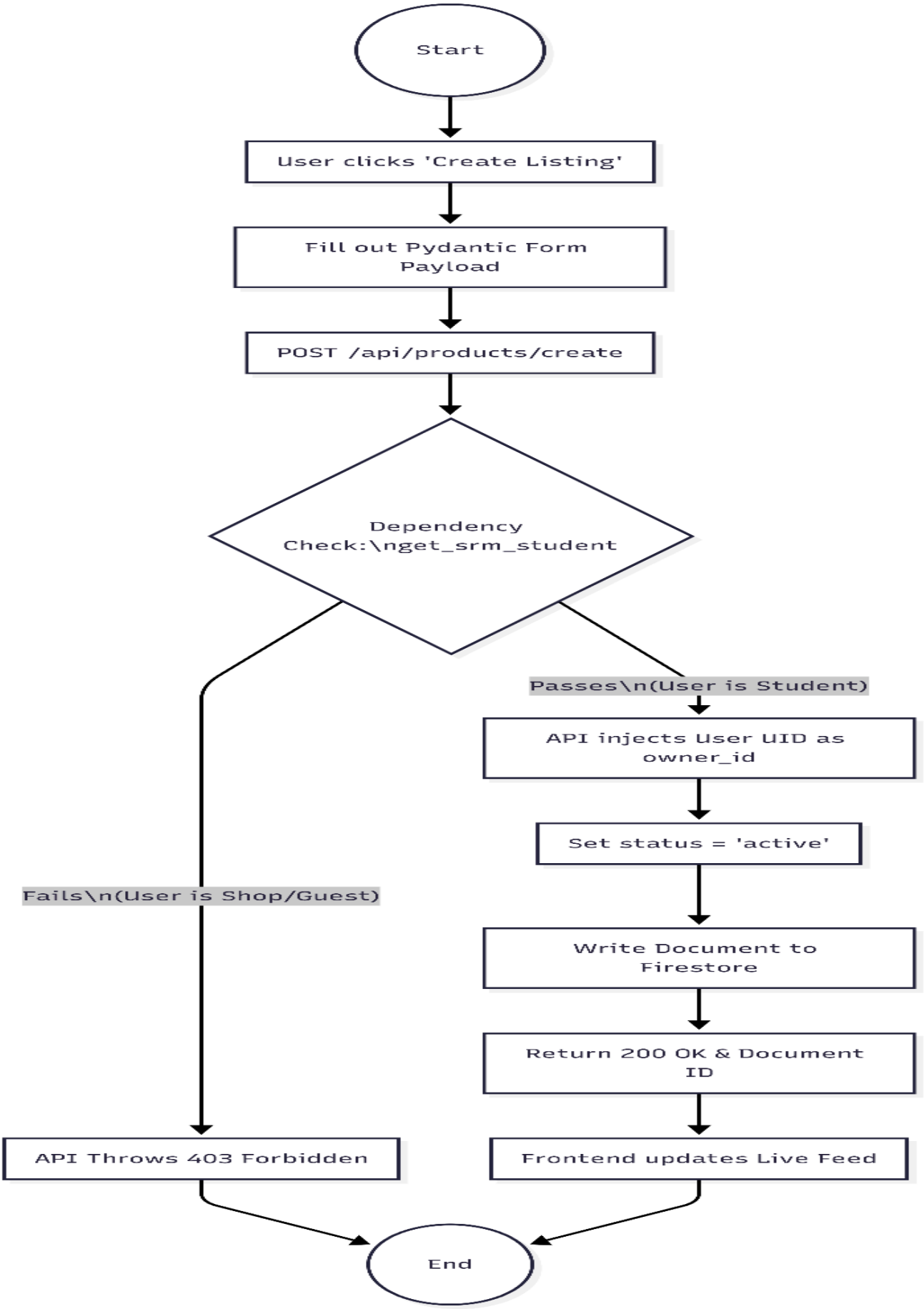

Student (Seller)
Shop (Buyer)
FastAPI Backend
Firestore DB
Super Admin
PHASE 1: Authentication & Profile Sync
1 POST /api/users/sync (Sends Google Token)
2 Check if UID exists in Users Collection
3 User not found
4 Create Profile (Extracts @srmist.edu.in -> Role: student)
5 200 OK (Profile Synced)
PHASE 2: Market Creation (P2P)
6 POST /api/products/create (Item: Used Laptop)
Security: get_srm_student checks Door 2
7 Validate domain is @srmist.edu.in
8 Save Listing (Status: "active")
9 200 OK (Ad Posted Successfully)
PHASE 3: Browsing & Cross-Market Bidding
10 GET /api/products/live
Security: get_marketplace_user checks Door 4
11 Query Listings where Status == "active"
12 Returns "Used Laptop"
13 Displays Laptop on Live Feed
14 POST /api/chat/initiate (Bid: ₹40,000)
15 Verify Target Owner Role (Student = Allowed)
16 Create Chat Room & Save Pending Message
17 200 OK (Room Created)
PHASE 4: Negotiation & Acceptance
18 GET /api/chat/inbox
19 Return Rooms where participant == Student.uid
20 Show Inbox (New Bid Notification)
21 POST /api/chat/{room_id}/messages/{msg_id}/accept
Security: Verify Student actually owns the Laptop
22 UPDATE Listing Status = "sold"
23 UPDATE Message bid_status = "accepted"
24 200 OK (Deal Closed & Removed from Feed)
PHASE 5: Post-Sale Support & Admin God Mode
25 POST /api/support/create (Ticket: "Help with billing")
26 Save Support Ticket (Status: "open")
27 GET /api/support/admin/all
28 Returns all open campus tickets
29 Displays Admin Dashboard
30 POST /api/support/admin/{ticket_id}/reply (Status: "resolved")
31 Update Ticket Status & Attach Admin Response
32 200 OK (Ticket Closed)
Student (Seller)
Shop (Buyer)
FastAPI Backend
Firestore DB
Super Admin

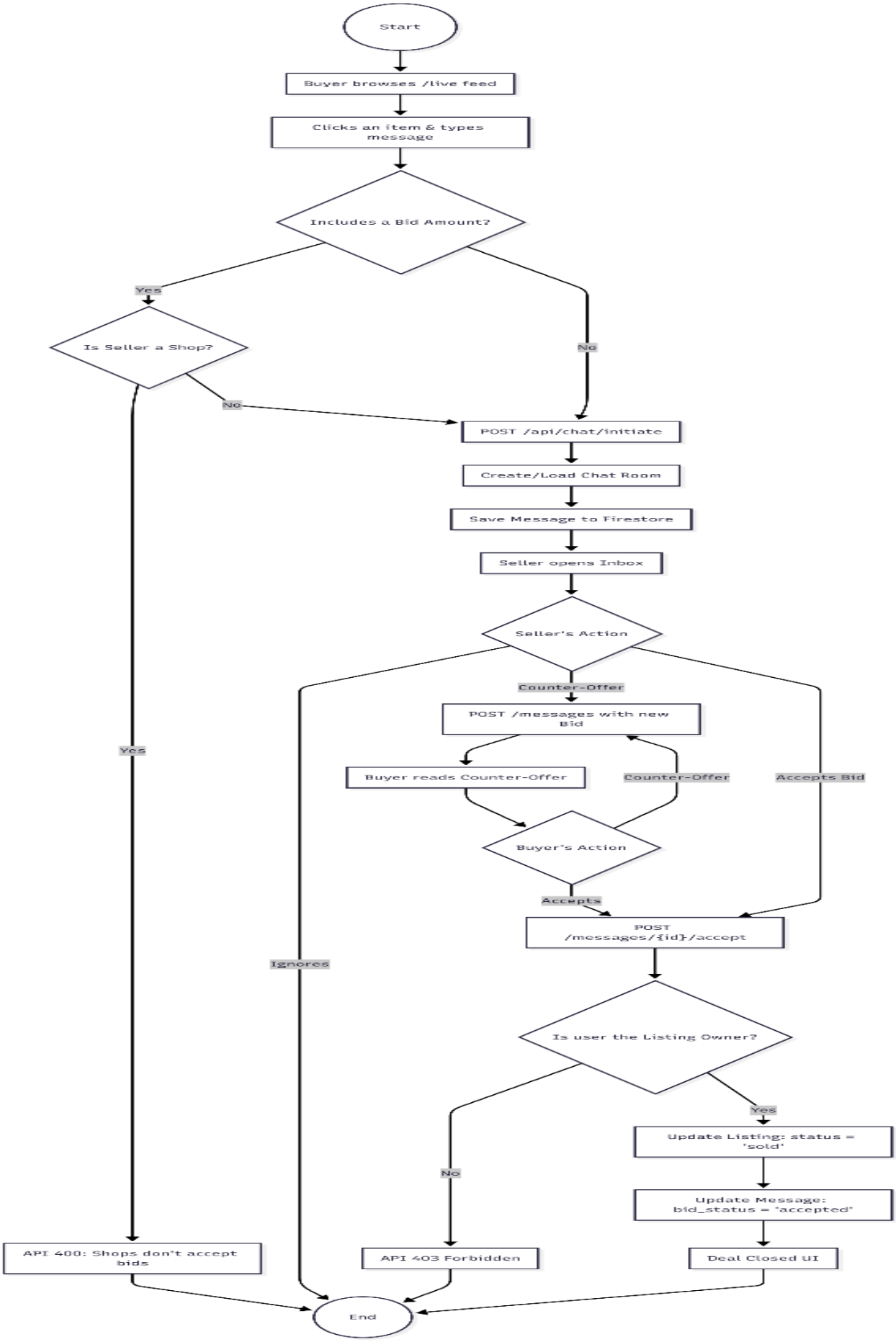

1. Account & Role Lifecycle
Logs in with Google
Guest
Submits Shop Application
Admin Rejects
Syncs with @srmist.edu.in
PendingApproval
Admin Approves
(is_verified = true)
Student
VerifiedShop


2. Listing & Negotiation Lifecycle
User creates listing
Active
Buyer initiates chat/bid
Bid rejected / Chat
abandoned
Negotiating
Buyer sends offer
PendingBid
Seller replies with new
price
Buyer replies with new price
CounterOffer
Admin moderates or User
deletes
Seller clicks 'Accept'
Sold
Deleted
Hidden from live feed
Hidden from live feed


3. Support Ticket Lifecycle
User submits issue
Open
Admin assigns ticket to
themselves
InProgress
Admin resolves immediately
Admin sends response
Resolved


Start
User logs in via Google Auth
Frontend calls POST /api/users/sync
API extracts Email & UID
Email ends with\n@srmist.edu.in?
Yes
No
Assign Role: 'student'
Route to Student Dashboard
Assign Role: 'guest'
Does user apply\nfor a Shop?
No
Yes
Route to Guest/Public View
POST /api/shops/create
Wait for Admin Review
Admin Approves?
No
Yes
Assign Role: 'shop'
Route to Business Seller Center
End


Start
User clicks 'Create Listing'
Fill out Pydantic Form Payload
POST /api/products/create
Dependency Check:\nget_srm_student
Passes\n(User is Student)
API injects User UID as owner_id
Set status = 'active'
Write Document to Firestore
Return 200 OK & Document ID
Frontend updates Live Feed
Fails\n(User is Shop/Guest)
API Throws 403 Forbidden
End


Start
Buyer browses /live feed
Clicks an item & types message
Includes a Bid Amount?
Yes
No
Is Seller a Shop?
No
Yes
POST /api/chat/initiate
Create/Load Chat Room
Save Message to Firestore
Seller opens Inbox
Seller's Action
Counter-Offer
POST /messages with new Bid
Buyer reads Counter-Offer
Counter-Offer
Accepts Bid
Buyer's Action
Accepts
POST /messages/{id}/accept
Ignores
Is user the Listing Owner?
Yes
No
Update Listing: status = 'sold'
Update Message: bid_status = 'accepted'
API 400: Shops don't accept bids
API 403 Forbidden
Deal Closed UI
End

Lab 9: Sequence Diagram, State Diagram, and Deployment Diagram Descriptions

1. Sequence Diagram Description (Time-Ordered Flow)

While interaction diagrams focus on structure, Sequence Diagrams map exactly how objects communicate over time. This description details the most technically complex feature of your platform: the real-time P2P bidding engine.

Scenario: Student A (Buyer) negotiates with Student B (Seller) over a used textbook.

1. **Student A (Client)** taps "Make Offer" on a textbook listing.
2. **Student A (Client)** sends a request to the **FastAPI Gateway** to initialize a chat.
3. The **FastAPI Gateway** validates the tokens, creates a negotiation_room document in the **Firebase Database**, and returns the room_id to Student A.
4. **Student A (Client)** and **Student B (Client)** both open a direct **Firestore Real-time Listener** subscribing to that specific room_id (bypassing the FastAPI server).
5. **Student A (Client)** submits a bid of "₹400". The client writes this directly to the **Firebase Database**.
6. The **Firebase Database** instantly pushes the "₹400" update to **Student B (Client)** via the open listener.
7. **Student B (Client)** clicks "Accept Offer."
8. **Student B (Client)** sends an HTTP POST request to the **FastAPI Gateway** (/api/chat/accept_bid).
9. The **FastAPI Gateway** validates the acceptance, instructs the **Firebase Database** to lock the negotiation_room and change the textbook's status to "Sold".
10. The **Firebase Database** returns a success confirmation to the gateway, closing the loop.

2. State Diagram Description

This state chart maps the lifecycle of the **Negotiation Room** (the chat session) during the P2P bidding process.

Entity: The Negotiation Room Document

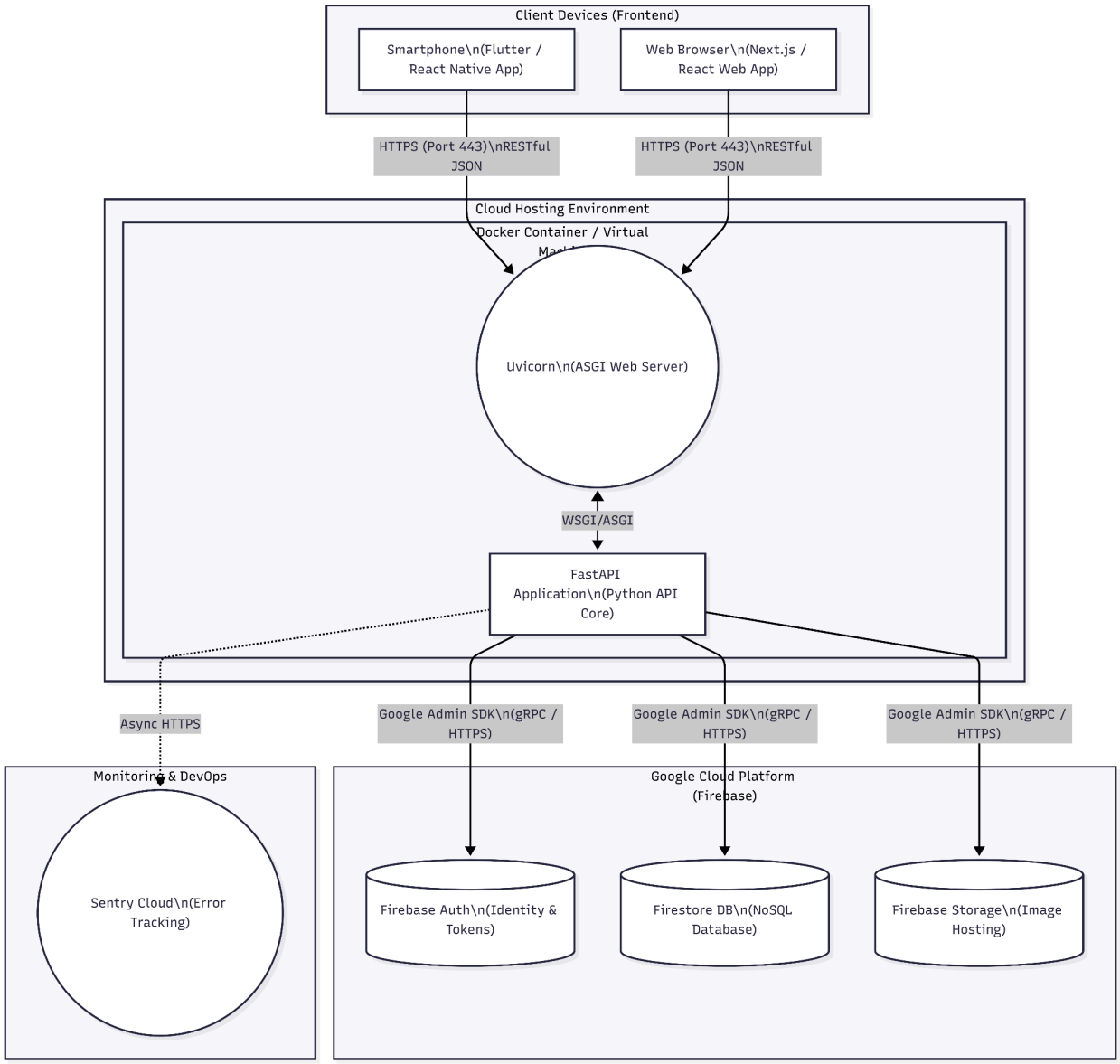
- **Initial State [Start]** $\rightarrow$ Transitions immediately to **Active**.
  - *Trigger:* A buyer initiates a chat on a seller's item. The room document is created.
- **State: Active**
  - *Internal Action:* Users can send messages and update the latest_bid integer.
  - *Transition A:* $\rightarrow$ **Locked (Accepted)**.
    - *Trigger:* The seller explicitly accepts the current bid. The backend freezes the chat to prevent further negotiation and generates a digital receipt.
  - *Transition B:* $\rightarrow$ **Abandoned**.
    - *Trigger:* Neither user sends a message for 7 days, or the item is sold to someone else in a different chat room.
- **Final State [End]**

    - Reached when the room becomes **Locked** or **Abandoned**. Listeners are detached to save database read quotas.

3. Deployment Diagram Description

The deployment diagram shows the physical (or cloud) hardware nodes where your software actually runs, and the communication protocols connecting them. This maps your production-ready architecture.

- **Node 1: User Device (Execution Environment: Browser / Mobile OS)**
    - *Components:* Runs the compiled React/Next.js or Flutter Client Application.
    - *Protocols:* Communicates outbound via HTTPS (REST API calls) and WSS (Secure WebSockets for Firebase).
- **Node 2: Cloud Application Server (Execution Environment: Docker Container / VM)**
    - *Components:* Hosts the Uvicorn ASGI Server running the FastAPI python application. Includes the .env vault for secure configuration.
    - *External Integration:* Integrates directly with Sentry via a background worker for catching and logging 500-level exception crashes.
    - *Protocols:* Receives HTTPS traffic from Node 1. Communicates with Node 3 via the secure Firebase Admin SDK (gRPC/HTTPS).
- **Node 3: Database as a Service (Execution Environment: Google Cloud / Firebase)**
    - *Components:* Hosts the Firestore NoSQL Database (managing collections and real-time document listeners) and the Google Identity Platform (handling OAuth token generation).
    - *Protocols:* Pushes real-time updates directly to Node 1 via WSS.


Client Devices (Frontend)
Smartphone\n(Flutter / React Native App)
Web Browser\n(Next.js / React Web App)
HTTPS (Port 443)\nRESTful JSON
HTTPS (Port 443)\nRESTful JSON
Cloud Hosting Environment
Docker Container / Virtual Machine
Uvicorn\n(ASGI Web Server)
WSGI/ASGI
FastAPI Application\n(Python API Core)
Async HTTPS
Google Admin SDK\n(gRPC / HTTPS)
Google Admin SDK\n(gRPC / HTTPS)
Google Admin SDK\n(gRPC / HTTPS)
Monitoring & DevOps
Sentry Cloud\n(Error Tracking)
Google Cloud Platform (Firebase)
Firebase Auth\n(Identity & Tokens)
Firestore DB\n(NoSQL Database)
Firebase Storage\n(Image Hosting)